Gullak

Practise money decisions safely — make choices, see consequences, try again.

A single-player, offline-first mobile game that teaches Grade 11–12 students in Hyderabad to budget, save, understand investment risk and stay safe from scams, through short English/Telugu stories that use pretend money only.

The plan covers 21 missions across four pillars. Each topic has a study pack (theory, flashcards, quiz, recap) and a story mission. All amounts, accounts and assets are fictional. The app has no real transactions, needs no login and collects no personal financial data.

Target users	Grade 11–12 students (about ages 16–18), underserved communities, Hyderabad / Telangana
Platform	Android mobile app (Flutter + Dart), fully usable offline
Languages	English and Telugu (Telugu copy marked draft until reviewed by a fluent speaker)
Technology stack	Flutter + Dart · Flutter animations (Flame only if needed) · Riverpod · SQLite via sqflite · JSON content

2 Introduction

A. Problem statement

Students in Grades 11–12 start making real money decisions before most have had a safe, structured chance to practise them. They manage bus or Metro fares, canteen food, mobile recharges, exam and coaching costs, and sometimes pocket money. They also plan for higher education and a first salary, and receive UPI payment requests and "prize" messages. These choices have consequences: an over-spent week, a surprise expense with no buffer, a loan whose total repayment was never compared, or an OTP shared with a stranger.

For students from underserved communities in Hyderabad, the gap is less about access to definitions and more about *practice*. Classroom or booklet explanations of budgets, interest or scams stay abstract until the student has to apply them to a concrete, familiar situation. Learning resources also often assume reliable connectivity, English fluency, a personal device, or a family background with formal financial products. The problem statement asks for an experience that is simple and relatable, interactive and fun, and safe to explore. Students should be able to make mistakes and build confidence without real-world consequences, across four pillars: plan & budget, save & manage, understand investing, and stay financially safe.

B. Existing solutions

Several credible free or low-cost resources already exist. The table separates **publicly available information** from **our assessment** of how well each fits our specific audience. This is a fit analysis, not a claim that any product is ineffective.

Solution	What it offers	Our assessment for Grade 11–12 students in Hyderabad
NCFE programmes (promoted by RBI, SEBI, IRDAI, PFRDA)	A not-for-profit national body promoting financial education. It runs free workshops and materials, teacher training for classes VIII–X, and short videos including Indian Sign Language versions.	Authoritative content. Delivery is mainly workshop and material based, so it depends on a session being scheduled. It is not a self-paced game a student can replay offline.
RBI Financial Education (FAME booklet, Money Kumar / Raju series)	Pictorial booklets and awareness messages, including tailored content for school children, available in 13 regional languages for download.	Strong on language reach and credibility. The format is mostly read-only, with no decisions, simulated consequences or progress tracking.
Khan Academy — Financial Literacy	Free, self-paced course covering budgeting, saving, credit, investing, insurance, taxes and scams, with videos, articles and exercises.	High-quality and free. It is video-heavy, generally needs connectivity, and its examples are not localised to Telugu-speaking Hyderabad students.
Zogo	Gamified app for ages 13+ with short modules and quizzes. Users earn points redeemable for gift cards. It is usually distributed through partner credit unions using access codes.	Shows that gamified finance learning appeals to teens. It is built for the US market and partner-gated, and its rewards have real value. Our design avoids real-value rewards on purpose.
FamPay	Indian teen payments app and card. With parental consent, teens can make UPI and card payments. It uses gamified savings challenges with in-app coins.	Relevant Indian context. It is a real-money payment product, so learning happens with actual funds and needs a parent-funded account. Our app teaches through simulation only.

C. Proposed solution

Gullak is a lightweight Flutter app that turns each financial concept into a short story with a decision. A student picks English or Telugu and sees a clear "pretend money only" notice. They can optionally create a nickname-only local profile, so several students can share one phone. They then choose where to start. Each of the 21 topics, spread across the four pillars, offers two ways in that can be used in either order:

- **Study:** a short theory lesson with a worked fictional example, at least three tap-to-reveal flashcards, a quiz of at least three questions with an explanation for every option, and a recap with a common mistake to avoid.
- **Play:** a story mission (for example, "Neha has ₹500 of pretend money this week..."). The student allocates a fictional budget, handles a surprise expense, compares invented assets, or decides what to do with an OTP request. The virtual balance updates, the consequence is shown, and a non-judgemental "What this teaches" explanation follows. The student can always retry.

Difficulty and different starting points. Students can start at *Easy* (one decision, one goal, explicit hints), *Medium* (competing goals or a surprise event, less prompting) or *Advanced* (several goals, uncertainty, comparisons over time). They can also take an optional short level check that suggests a level. Earlier levels are never prerequisites. Students who start at Medium or Advanced are offered a skippable warm-up of earlier key concepts PROPOSED. Scam and PIN/OTP safety guidance is available from the home screen at every level.

Personalised practice budgets PROPOSED. Our product plan forbids asking students for real pocket money or household income. Instead, students pick a *fictional practice budget* preset (weekly or monthly), or "I don't receive regular pocket money", which uses a zero-based scenario. Budgeting missions use this preset as their starting virtual balance, so scenarios feel relevant without any personal disclosure.

D. Key differentiators and expected impact

Differentiator	How the design delivers it
Learn by deciding	Every mission requires a choice, shows its simulated consequence and explains the principle. Wrong choices lead to an explanation and a retry, never a penalty.
Local relevance and language	Hyderabad-context stories (bus/Metro, canteen, recharge, exam fees, UPI-style QR safety) in English and Telugu. Students can switch language without losing progress.
Accessibility and affordability	Free to distribute, offline-first and small enough for inexpensive Android phones. No login, no data plan, and supports shared devices with multiple local profiles.
Study and play for every topic	All 21 topics pair a study pack with a mission. Students can revise first or play first.
Safety and privacy by design	Fictional currency with no real value. Investing uses invented assets only ("simulation only — not financial advice"). Scam links are inert. No real money movement is possible, and no sensitive data is collected.
Non-coercive engagement	Private badges such as "Budget Planner" and "Scam Spotter". No leaderboards, streak pressure, loot boxes or progress loss.

Expected impact (not yet measured). We expect students to gain practice in recognising needs vs. wants, building and adjusting a budget, valuing a buffer, comparing borrowing costs, understanding that higher possible returns come with higher risk, and refusing to share PINs/OTPs. The app is *designed for* distribution to 1,500+ students through NGOs or schools. Our prototype checks will use scripted and synthetic data only. We make no claim of measured learning gains or verified scale.

3 High-Level System Architecture

Gullak is a **self-contained, offline-first client application**. The product plan rules out mandatory login, cloud-dependent gameplay and identifying analytics. So there is **no backend server, remote API or authentication service** in the architecture. All game logic, content and progress live on the device. Components communicate through in-process Dart calls and asynchronous storage APIs, never a network protocol.

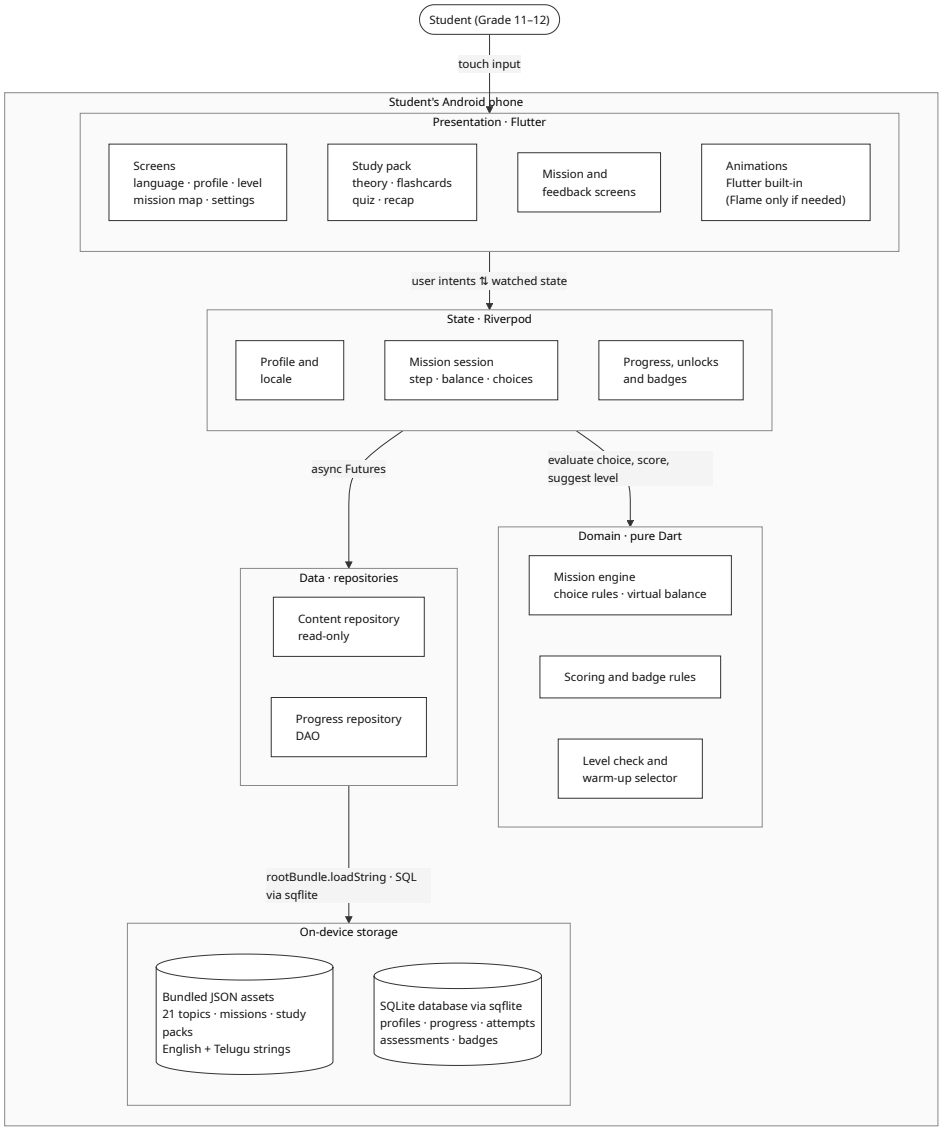


Figure 1. High-level architecture. All components run on the student's device. Arrows show the direction of requests and data flow.

Component	Purpose	Why this technology
Presentation layer (Flutter + Dart)	Screens for language, profile, level, mission map, study pack, mission, feedback and settings. Large touch targets, text scaling, one main action per screen.	One codebase compiles to a native Android app with consistent rendering on low-cost devices. Flutter's localisation support handles English and Telugu.
Animation layer (Flutter animations; Flame optional)	Balance counters, outcome reveals, flashcard flips, badge effects.	Built-in animations need no extra dependency. Flame is added only if a mission needs a true game loop, which keeps the app small.

State layer (Riverpod)	Holds the current profile and locale, the active mission session (step, virtual balance, choices), progress, unlock indicators and badges.	Compile-time-safe providers, testable without widgets, and a clear separation between UI and logic.
Domain layer (pure Dart)	*Mission engine* applies each choice's rule (balance change, scenario state, result). *Scoring and badge rules* compute points and awards. *Level check* suggests a level and selects warm-up cards.	Pure functions with no Flutter dependency, so every financial rule can be unit-tested with fixed inputs.
Content repository + JSON assets	Loads the 21 topic definitions, missions, choices, outcomes, quiz items, flashcards and bilingual strings bundled with the app.	JSON is readable by non-developers, so content and translations can be edited without code changes. Bundled assets work offline.
Progress repository + SQLite (sqflite)	Stores local profiles, mission progress, attempts and decisions, study progress, assessment scores and badges.	A relational store with transactions keeps progress consistent across app restarts. It supports several profiles on a shared device and runs fully offline.

Maintainability, scalability and reach

- **Maintainability:** a layered design (UI → Riverpod → domain → repositories). All 21 missions use one reusable story → choice → outcome → explanation template, so a new mission means new JSON, not new code.
- **Scalability for 1,500+ students:** each install is independent with no shared server, so adding students adds no server load. Distribution scale depends on packaging and installation (APK sideloading or store listing via NGOs). We do not claim tested performance at that scale. Optional anonymised aggregate reporting for NGOs is a possible *future* extension and is not part of this architecture.
- **Privacy:** no network permission is needed for gameplay. Data never leaves the device, and profiles hold only an optional nickname, grade and avatar.

4 Application Flow

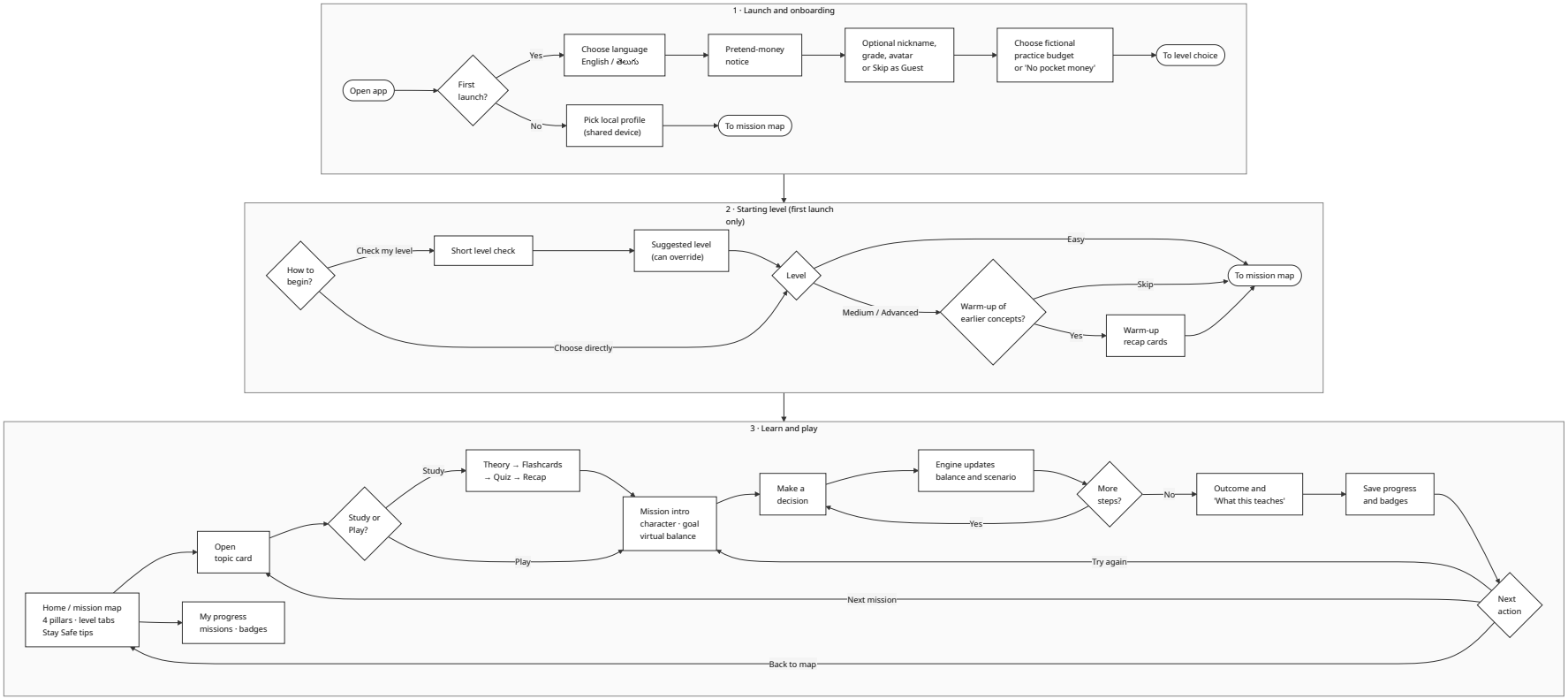


Figure 2. Student journey from first launch to mission completion and progress review.

Key decision points

- **First launch vs. returning:** language choice and the pretend-money notice appear on first launch. Returning students pick their local profile and go straight to the mission map.
- **Profile:** creating a profile is optional ("Skip as Guest"). Fields are limited to nickname, grade and avatar. The practice budget is a fictional preset, never real pocket money.
- **How to begin:** students choose Easy, Medium or Advanced directly, or take the short level check, whose suggestion they can override. Medium and Advanced starters are offered a skippable warm-up of earlier concepts.
- **Study or Play:** both routes are open for every topic. Study ends with an offer to play the mission.
- **Mission loop:** each decision is evaluated and the balance updates. Multi-step missions (Medium/Advanced) loop until all steps are done. Progress is saved before the student chooses Try again, Next mission or Back to map.

5 Database / Entity-Relationship Design

The data model has two parts that match the technology stack. **Content** (topics, missions, choices, quiz questions, badges) is read-only and stored in **bundled JSON files**. **Learner records** are written to **SQLite** through sqflite. In the diagram, content entities are marked "JSON asset". SQLite tables refer to them through string IDs (for example, `mission_id = "M01"`). These are logical references checked by the repository layer, not database foreign-key constraints.

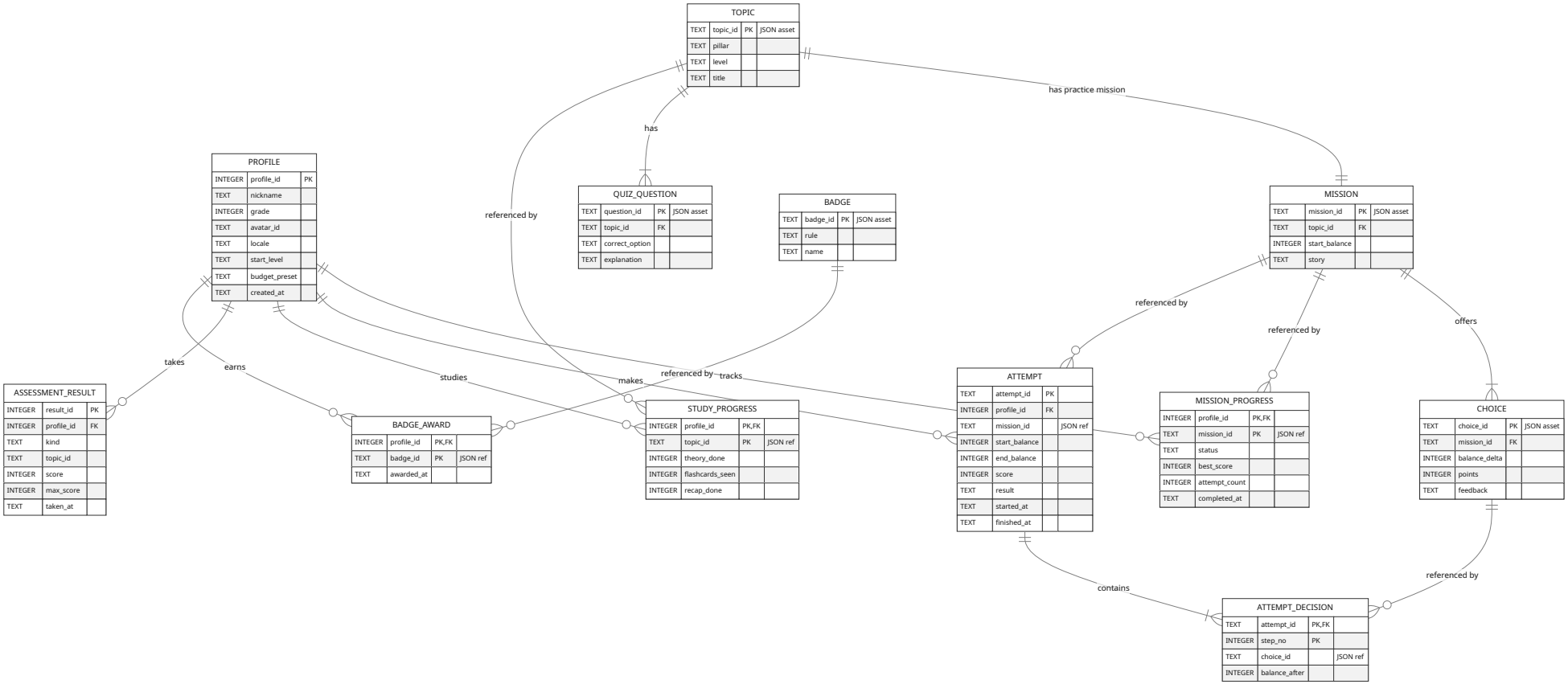


Figure 3. Logical ER diagram. PROFILE, MISSION_PROGRESS, ATTEMPT, ATTEMPT_DECISION, STUDY_PROGRESS, ASSESSMENT_RESULT and BADGE_AWARD are SQLite tables. The others are JSON content entities.

Key relationships and design decisions

- **PROFILE → everything:** every learner record belongs to one local profile. Deleting a profile cascades to its rows, which supports the "clear local progress" setting and shared devices.

- **MISSION_PROGRESS** (composite key: profile + mission) holds one summary row per mission: status, best score and attempt count. It drives the mission map and completion indicators without scanning the attempt history.
- **ATTEMPT and ATTEMPT_DECISION** record each play-through and each step's choice and resulting virtual balance. The composite key (attempt_id, step_no) rejects duplicate submissions of the same step. An attempt with no `finished_at` marks an interrupted session that can be resumed or discarded.
- **STUDY_PROGRESS and ASSESSMENT_RESULT** track the study pack, level-check, quiz and end-check scores. This lets the app show each student a private before-and-after comparison.
- **Data minimisation:** balances are virtual integers in rupees taken from scenario data. No real income, pocket money, bank or contact details exist anywhere in the schema.
- **Proposed vs. final:** column names and types are our proposed schema. Timestamps are ISO-8601 TEXT. Status and result fields are short enumerations (for example, `not_started / in_progress / completed`). Optional profile fields are nullable. The JSON entity fields shown are the subset the database refers to. Full content files also hold flashcards, theory text and recap fields.

6 Sequence Diagram — One Game Session

Representative mission: **01 · Needs vs. Wants (Plan & Budget, Easy)**, demo scenario A from our product plan. Neha has ₹500 of pretend money. Essentials (transport ₹150, lunch ₹180, notebooks ₹70) total ₹400. Optional snacks (₹40) and a movie (₹120) would bring the total to ₹560. Because the architecture is client-side, the controller, engine and repositories below are in-app components, not remote services.

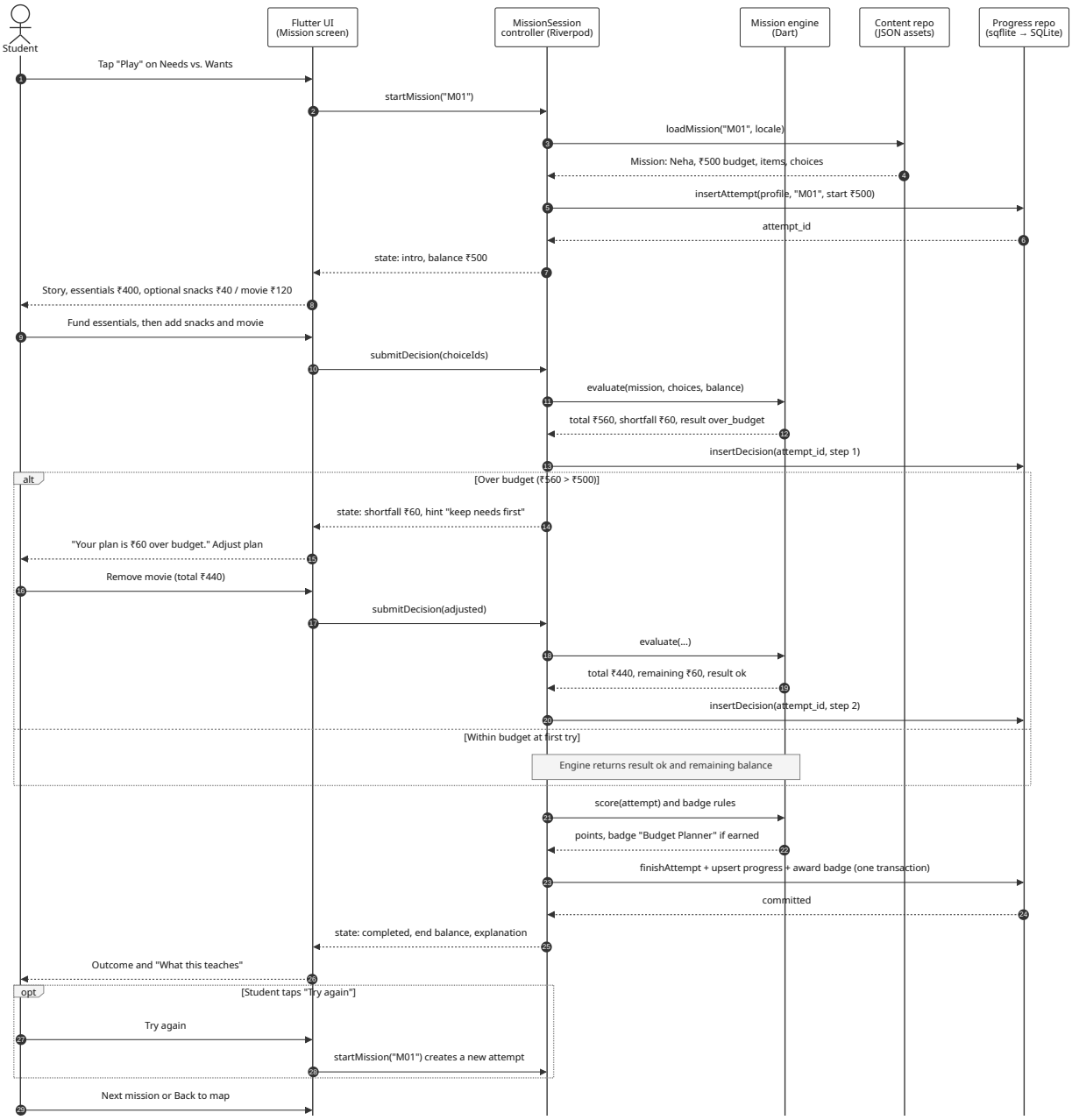


Figure 4. One complete session of "Needs vs. Wants", including the over-budget correction path and an optional retry.

Explanation. Starting the mission loads its definition from the JSON assets in the selected language and opens an ATTEMPT row. The student's decision goes to the Riverpod controller, which asks the pure-Dart engine to evaluate it against the mission rules. If the plan is over budget, the student sees the exact ₹60 shortfall and adjusts. Each step is recorded as an ATTEMPT_DECISION. When the mission ends, scoring and badge rules run. The attempt result, progress summary and any badge are committed in **one SQLite transaction**, so an interruption cannot leave progress half-saved. The student then sees the outcome and explanation, and can retry (which creates a new attempt) or move on.

7 Planned Testing Methodology

> **Scope note.** Our product plan rules out recruiting real students for testing, interviews or pilots during the hackathon. All testing below uses automated tests, internal scripted walkthroughs by the team, and synthetic learner data. All targets are *proposed targets*. No results exist yet.

A. Functional testing

What: onboarding, profile creation and skip, practice-budget preset, level selection and level check, study pack, mission access, scoring, badges, persistence. *How:* Flutter widget tests, Riverpod provider tests, and scripted manual walkthroughs on an emulator and one low-end Android phone.

Representative test case	Expected outcome / acceptance criterion
Fresh install → Telugu → Skip profile → Advanced → Skip warm-up.	Mission map opens with the Advanced tab active. Easy and Medium missions stay accessible (no gatekeeping).
Finish mission 19 (Secret PIN Safety) with the safe choice, then restart the app.	Mission shows "completed". The "Scam Spotter" badge is listed once.
Answer a quiz question wrong, then retry.	The explanation for the chosen option is shown. Retrying does not reduce saved progress.

B. Financial simulation testing

What: budget totals, shortfalls, savings contributions, borrowing cost, compound interest, simulated investment outcomes. *How:* table-driven unit tests of the pure-Dart engine using the worked examples in our content as test fixtures, plus a JSON content validator script.

Representative test case	Expected outcome / acceptance criterion
Needs vs. Wants: ₹500 budget, essentials ₹400 + snacks ₹40 + movie ₹120.	Total ₹560, shortfall ₹60, result `over_budget`. Removing the movie gives ₹440, remaining ₹60.
Interest Insights: ₹100 at a fictional 10% annual compound rate for 2 years. Borrowing: ₹500 item, ₹560 repayment.	₹121, and a borrowing cost of ₹60.
Boundary cases: zero-based "no pocket money" preset, a spend exactly equal to the budget, and a choice that would make the balance negative.	The zero preset shows a saving-free path without shame text. An exact spend gives remaining ₹0, not over budget. The balance never displays below ₹0 unless the mission defines a shortfall state.

Proposed target: 100% of authored worked examples pass. The content validator reports zero missions with missing feedback text, missing Telugu strings or rule references that point to nothing.

C. Integration testing

What: UI ↔ Riverpod ↔ engine ↔ repositories; JSON asset loading; SQLite reads and writes; profile isolation. There is no backend API or authentication to test, so backend API and session tests are not applicable. Profile isolation testing takes their place. *How:* Flutter `integration_test` on an emulator with a real sqflite database, plus repository tests against an in-memory database.

Representative test case	Expected outcome / acceptance criterion
Play a full mission end to end through the UI. Then query SQLite.	One ATTEMPT with `finished_at` set, matching ATTEMPT_DECISION rows, and an updated MISSION_PROGRESS row.

Two profiles on one device each complete different missions.	Each profile's map shows only its own progress. Deleting profile A leaves profile B unchanged.
Switch language from English to Telugu mid-journey.	All strings reload from the Telugu JSON file. Progress and the current mission state are kept.

D. Usability and accessibility testing

What: navigation clarity, instruction and feedback wording, suitability for ages 16–18, readability, responsiveness, bilingual parity. *How:* heuristic review by the team against a checklist (one action per screen, no dead ends, labelled icons, safe/unsafe not shown by colour alone), checks at system font scales 100% and 130%, small and large screen emulators, and Flutter's accessibility guideline tests. Real-student usability testing is out of scope. Before rollout, the deploying NGO or school would run it.

Representative test case	Expected outcome / acceptance criterion
Walk the main path in Telugu at 130% font scale on a 5-inch emulator.	No clipped prompts or overlapping buttons. Every screen has a visible continue or back action. Telugu text is shown with a "draft" marker.

E. Learning-outcome evaluation (instrument design)

What: whether gameplay is associated with better understanding of concepts and consequences. *How:* the app will include an optional pre-play level check and an optional end-of-pillar check built from parallel question sets. Scores are stored in ASSESSMENT_RESULT and shown privately to the student as a before-and-after comparison. Each pillar includes "why" questions that ask the student to pick the consequence of a decision. During the hackathon we will verify the *instrument* only, using synthetic personas (for example, a "careful saver" and an "impulsive spender" answer script) to confirm scoring and comparison logic. Any real evaluation with students would be designed and consented by the deploying organisation.

Representative test case	Expected outcome / acceptance criterion
A synthetic persona scores 2/6 on the level check, then 5/6 on the end-of-pillar check.	Progress screen shows "2 → 5 of 6" for that pillar. No class or peer comparison is shown anywhere.

Proposed metrics for future real-world use (not measured here): average change between pre- and post-check scores per pillar, share of students choosing the safe action in scam missions, and mission completion and retry rates.

F. Reliability, security and edge-case testing

What: invalid or empty inputs, interrupted sessions, repeated submissions, local data protection, error handling. *How:* unit and integration tests, forced app kills during missions, airplane-mode runs, inspection of the app manifest and the stored database.

Representative test case	Expected outcome / acceptance criterion
Kill the app after step 1 of a two-step mission, then relaunch.	The unfinished attempt is detected and the student can resume or restart. MISSION_PROGRESS is not marked completed.
Double-tap "Submit" on one decision.	Exactly one ATTEMPT_DECISION row for that step. The balance changes only once.
Empty or 40-character nickname; corrupted or missing mission JSON entry.	An empty nickname falls back to "Guest" and a long one is trimmed to the limit. A bad content entry shows a friendly "mission unavailable" card instead of crashing.
Airplane mode from first launch; scam-mission mock link tapped.	Full main path works offline. The mock link opens no browser or external intent. No network request is made.

Inspect onboarding screens and the SQLite file.	No full name, phone, school, location, income or real pocket-money field exists.

How results feed back into development

Failures are logged against the P0/P1/P2 priority list in our build plan. P0 issues block the demo and are fixed first: any arithmetic error, unsafe scam guidance, real-looking transaction, data-loss bug or offline failure. Content-validator failures go back to the JSON authors, and Telugu issues are queued for review by a fluent speaker. In the final submission we will report exactly which checks were run, which missions are playable versus specified only, and what remains unverified.